Software Construction: Core Practices

Software construction is the detailed creation of working software through coding, verification, unit testing, integration testing, and debugging.

Key Principles

- **Minimize complexity:** favor simple, readable code over clever code.
- **Anticipate change:** isolate likely changes behind stable interfaces.
- **Construct for verification:** write code that is easy to test and review.
- **Reuse and standards:** follow team conventions and reuse proven components.

Defensive Programming

Validate inputs at boundaries, use assertions for conditions that should never occur, and handle errors explicitly rather than ignoring them. Fail fast so bugs surface near their cause.

Example: Input Validation

```
def withdraw(balance, amount):
    if amount <= 0: raise ValueError('amount must be positive')
    if amount > balance: raise ValueError('insufficient funds')
    return balance - amount
```

Testing and Integration in Construction

Construction does not end when code compiles. Testing and integration confirm that the pieces work alone and together.

Levels of Testing

- **Unit tests:** verify one function or class in isolation; fast and numerous.
- **Integration tests:** verify that modules and services interact correctly.
- **Regression tests:** rerun after changes to catch reintroduced bugs.

Integration Strategies

Incremental integration adds and tests components in small steps, so failures are easy to locate. **Big-bang integration** combines everything at once and is riskier because defects are hard to trace. Continuous integration automates builds and tests on every commit.

Code Review and Refactoring

Peer review catches defects early and spreads knowledge. Refactor in small steps, backed by tests, to improve structure without changing behavior. Common triggers include duplicated code, long methods, and unclear names.